INTEGRASI FIRMWARE ESP32 DAN LIBRARY PYTHON UNTUK PENGENDALIAN AKTUATOR PADA KIT IMCLAB

Dosen Pengampu:

Dr. Basuki Rahmat, S.Si, M.T

Disusun oleh:

M. Wifaqul Azmi 23081010246

MIKROKONTROLER A – 081
PROGRAM STUDI INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS PEMBANGUNAN NASIONAL " VETERAN"
JAWA TIMUR
2025

imclab_firmware.ino

```
/*
  iMCLab Internet-Based Motor Control Lab
  Firmware FIXED: LED + Motor OP + (Optional) RPM
  December 2025
*/

#include <Arduino.h>

// ===== constants =====
const String vers = "1.1";
const int baud = 115200;
const char sp = ' ';
const char nl = '\n';

// ===== pin numbers on iMCLab Shield (sesuai kode kamu) =====
const int motor1Pin1 = 27;   // IN1
const int motor1Pin2 = 26;   // IN2
const int enable1Pin = 12;   // EN (PWM motor)
const int pinLED     = 2;    // LED

// Encoder pin (RPM)
const int pin_rpm = 13;      // input encoder

// ===== PWM properties =====
const int freq = 30000;
const int pwmChannel = 1;
const int pwmresolution = 8;   // 0..255

const int ledChannel = 0;
const int ledresolution = 8;   // 0..255

// ===== RPM variables =====
volatile unsigned long pulseCount = 0;
unsigned long last_pulse_count = 0;
unsigned long last_rpm_time = 0;

float rpm = 0;
float rpm_filtered = 0;

// Ubah ini kalau disk encoder kamu beda
// Contoh dosen: holes = 2 (2 lubang/slot per putaran)
const int PULSES_PER_REV = 2;

// ===== serial parsing variables =====
char Buffer[64];
String cmd;
double pv = 0;

// ===== ISR encoder =====
void IRAM_ATTR onPulse() {
  pulseCount++;
}

void parseSerial() {
  // baca sampai newline
  int ByteCount = Serial.readBytesUntil(nl, Buffer, sizeof(Buffer));
  (void)ByteCount;
```

```
  String read_ = String(Buffer);
  memset(Buffer, 0, sizeof(Buffer));

  // pisahkan command dan data
  int idx = read_.indexOf(sp);
  if (idx < 0) {
    cmd = read_;
    pv = 0;
  } else {
    cmd = read_.substring(0, idx);
    String data = read_.substring(idx + 1);
    data.trim();
    pv = data.toFloat();
  }

  cmd.trim();
  cmd.toUpperCase();
}

void calculateRPM() {
  unsigned long now = millis();
  unsigned long dt = now - last_rpm_time;

  if (dt >= 200) {  // update tiap 200ms biar responsif
    noInterrupts();
    unsigned long p = pulseCount;
    interrupts();

    unsigned long dp = p - last_pulse_count;

    float rev = (float)dp / (float)PULSES_PER_REV;
    float rps = rev / (dt / 1000.0f);
    rpm = rps * 60.0f;

    // low-pass filter sederhana
    rpm_filtered = 0.7f * rpm_filtered + 0.3f * rpm;

    last_pulse_count = p;
    last_rpm_time = now;
  }
}

void setMotorPercent(float percent) {
  percent = constrain(percent, 0.0f, 100.0f);

  // arah default forward
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  int duty = (int)(percent * 255.0 / 100.0);
  duty = constrain(duty, 0, 255);

  ledcWrite(pwmChannel, duty);
}

void setLEDPercent(float percent) {
  percent = constrain(percent, 0.0f, 100.0f);
```

```
  int duty = (int)(percent * 255.0 / 100.0);
  duty = constrain(duty, 0, 255);

  ledcWrite(ledChannel, duty);
}

void dispatchCommand() {
  if (cmd == "OP") {
    // motor PWM 0..100%
    setMotorPercent((float)pv);
    Serial.println((float)pv);
  }
  else if (cmd == "RPM") {
    calculateRPM();
    Serial.println(rpm_filtered);
  }
  else if (cmd == "V" || cmd == "VER") {
    Serial.println("iMCLab Firmware Version " + vers);
  }
  else if (cmd == "LED") {
    setLEDPercent((float)pv);
    Serial.println((float)pv);
  }
  else if (cmd == "X") {
    ledcWrite(pwmChannel, 0);
    Serial.println("Stop");
  }
  else {
    Serial.println("ERR");
  }
}

// ===== arduino startup =====
void setup() {
  Serial.begin(baud);
  delay(1500);
  Serial.println("READY");

  // motor direction pins
  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  digitalWrite(motor1Pin1, LOW);
  digitalWrite(motor1Pin2, LOW);

  // motor PWM setup (PENTING: attach ke enable1Pin, bukan pin_rpm)
  ledcSetup(pwmChannel, freq, pwmresolution);
  ledcAttachPin(enable1Pin, pwmChannel);
  ledcWrite(pwmChannel, 0);

  // LED PWM setup
  ledcSetup(ledChannel, freq, ledresolution);
  ledcAttachPin(pinLED, ledChannel);
  ledcWrite(ledChannel, 0);

  // encoder interrupt
  pinMode(pin_rpm, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(pin_rpm), onPulse, RISING);

  last_rpm_time = millis();
```

```
}

// ===== main loop =====
void loop() {
  // update RPM in background
  calculateRPM();

  // handle serial command (blocking readBytesUntil will wait timeout if no data,
  // jadi lebih aman cek dulu tersedia)
  if (Serial.available()) {
    parseSerial();
    dispatchCommand();
  }
}
```

imclab.py

```
import sys
import time
import numpy as np
try:
    import serial
except:
    import pip
    pip.main(['install','pyserial'])
    import serial
from serial.tools import list_ports

class iMCLab(object):

    def __init__(self, port=None, baud=115200):
        port = self.findPort()
        print('Opening connection')
        self.sp = serial.Serial(port=port, baudrate=baud, timeout=2)
        self.sp.flushInput()
        self.sp.flushOutput()
        time.sleep(3)
        print('iMCLab connected via Arduino on port ' + port)

    def findPort(self):
        found = False
        for port in list(list_ports.comports()):
            # Arduino Uno
            if port[2].startswith('USB VID:PID=16D0:0613'):
                port = port[0]
                found = True
            # Arduino HDuino
            if port[2].startswith('USB VID:PID=1A86:7523'):
                port = port[0]
                found = True
            # Arduino Leonardo
            if port[2].startswith('USB VID:PID=2341:8036'):
                port = port[0]
                found = True
            # Arduino ESP32
            if port[2].startswith('USB VID:PID=10C4:EA60'):
                port = port[0]
```

```
                found = True
            # Arduino ESP32 - Tipe yg berbeda
            if port[2].startswith('USB VID:PID=1A86:55D4'):
                port = port[0]
                found = True
        if (not found):
            print('Arduino COM port not found')
            print('Please ensure that the USB cable is connected')
            print('--- Printing Serial Ports ---')
            for port in list(serial.tools.list_ports.comports()):
                print(port[0] + ' ' + port[1] + ' ' + port[2])
            print('For Windows:')
            print('  Open device manager, select "Ports (COM & LPT)"')
            print('  Look for COM port of Arduino such as COM4')
            print('For MacOS:')
            print('  Open terminal and type: ls /dev/*.')
            print('  Search for /dev/tty.usbmodem* or /dev/tty.usbserial*. The
port number is *.')
            print('For Linux')
            print('  Open terminal and type: ls /dev/tty*')
            print('  Search for /dev/ttyUSB* or /dev/ttyACM*. The port number is
*.')
            print('')
            port = input('Input port: ')
            # or hard-code it here
            #port = 'COM3' # for Windows
            #port = '/dev/tty.wchusbserial1410' # for MacOS
        return port

    def stop(self):
        return self.read('X')

    def version(self):
        return self.read('VER')

    @property
    def RPM(self):
        self._RPM = float(self.read('RPM'))
        return self._RPM

    def LED(self,pwm):
        pwm = max(0.0,min(100.0,pwm))/2.0
        self.write('LED',pwm)
        return pwm

    def op(self,pwm):
        pwm = max(0.0,min(100.0,pwm))
        self.write('op',pwm)
        return pwm

    # save txt file with data and set point
    # t = time
    # u = Output Controller
    # y = RPM
    # sp = setpoints
    def save_txt(self,t,u,y,sp):
        data = np.vstack((t,u,y,sp))  # vertical stack
        data = data.T                 # transpose data
        top = 'Time (sec), Ouput Controller (%), ' \
```

```
          + 'RPM (rpm), ' \
          + 'Set Point (rpm)'
        np.savetxt('data.txt',data,delimiter=',',header=top,comments='')

    def read(self,cmd):
        cmd_str = self.build_cmd_str(cmd,'')
        try:
            self.sp.write(cmd_str.encode())
            self.sp.flush()
        except Exception:
            return None
        return self.sp.readline().decode('UTF-8').replace("\r\n", "")

    def write(self,cmd,pwm):
        cmd_str = self.build_cmd_str(cmd,(pwm,))
        try:
            self.sp.write(cmd_str.encode())
            self.sp.flush()
        except:
            return None
        return self.sp.readline().decode('UTF-8').replace("\r\n", "")

    def build_cmd_str(self,cmd, args=None):
        """
        Build a command string that can be sent to the arduino.

        Input:
            cmd (str): the command to send to the arduino, must not
                contain a % character
            args (iterable): the arguments to send to the command
        """
        if args:
            args = ' '.join(map(str, args))
        else:
            args = ''
        return "{cmd} {args}\n".format(cmd=cmd, args=args)

    def close(self):
        try:
            self.sp.close()
            print('Arduino disconnected successfully')
        except:
            print('Problems disconnecting from Arduino.')
            print('Please unplug and reconnect Arduino.')
        return True
```

PROGRAM 1

1. Nama Program

Program Pengendalian Kecerahan LED

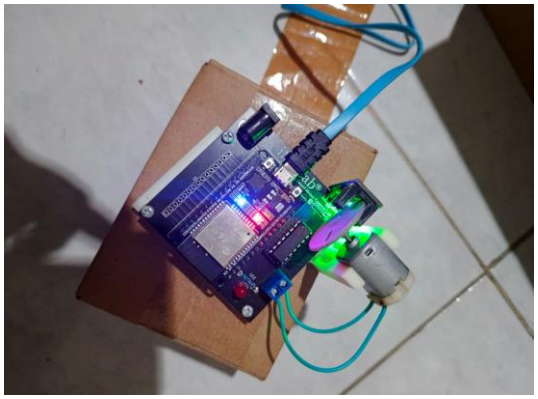
2. Kode Program

```
import imclab, time


a = imclab.iMCLab()
```

```
print(a.version())

for i in range(0, 101, 10):
    print("LED", i)
    a.LED(i)
    time.sleep(0.3)

for i in range(100, -1, -10):
    print("LED", i)
    a.LED(i)
    time.sleep(0.3)

a.LED(0)
a.close()
```

```
import imclab
import time

a = imclab.iMCLab()     # harusnya auto detect COM5
print(a.version())      # pakai kurung ya

print("LED ON")
a.LED(100)
time.sleep(1)

print("LED OFF")
a.LED(0)

a.close()
```

Hasil Program



PROGRAM 2

1. Nama Program

Program Kontrol Motor

2. Kode Program

```
import imclab, time

a = imclab.iMCLab()
print(a.version())

p = 80
print("Set OP", p, "reply:", a.op(p))
time.sleep(8)

print(a.stop())
a.close()
```

3. Hasil Program